HPCS EXECUTIVE ARCHITECTURE & EXECUTION REPORT

Parallelization of Pigeonhole Sort for Efficient Data Sorting

Bakkesh_HPCS Suite: Execution Workflow, Algorithmic Flowcharts & Advanced Enhancements

Department of Computer Science & Engineering | High Performance Computing Systems Laboratory

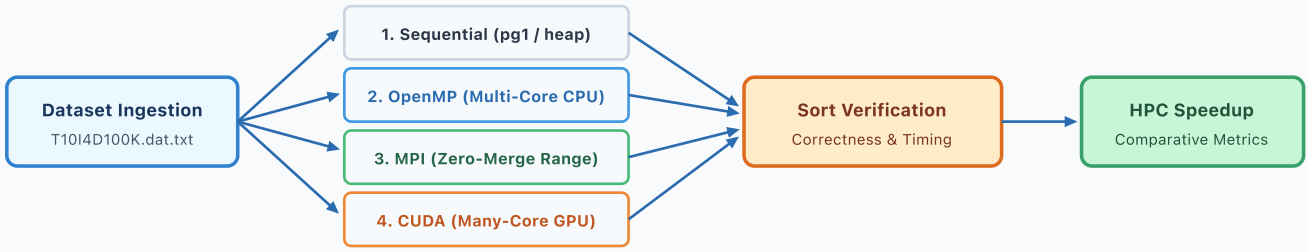
1. Project Mission & What We Are Going to Execute

The core mission of this project is to optimize and evaluate **Pigeonhole Sort**—a linear non-comparison distribution algorithm with theoretical time complexity `&mathcal;O(n + Range)`—across diverse high-performance parallel architectures. While sequential CPU sorting degrades when sorting massive datasets due to single-threaded compute limits, parallelizing via multi-core CPUs, distributed clusters, and GPUs unlocks dramatic acceleration.

The 4 Implementations in the `Bakkesh_HPCS` Execution Suite:

Module File	Computing Architecture	Execution Platform	Primary Function & Characteristics
`sequential_pigeonhole.c`	Single-Core CPU	MacBook / Linux (gcc)	Safe heap allocation (`calloc`), dynamic min/max detection, dynamic input buffering, monotonic time profiling.
`openmp_pigeonhole.c`	Multi-Core CPU (OpenMP)	MacBook / Linux / Colab	Multi-threaded shared-memory parallelism. Thread-local histograms, lock-free counting, and parallel prefix sum placement.
`mpi_range_partition.c`	Distributed Cluster (MPI)	OpenMPI / Google Colab	**Zero-Merge Range Partitioned Architecture**. Value-range slicing with zero post-merge overhead.
`cuda_scalable_pigeonhole.cu`	NVIDIA GPU (CUDA)	Google Colab (T4 GPU)	Multi-block grid-striding, global memory `atomicAdd`, device prefix scan, and parallel `atomicSub` scatter.

Figure 1: End-to-End System Execution Pipeline



2. Algorithmic Flowcharts: Baseline vs. What We Are Additionally Adding

Figure 2: MPI Architectural Comparison — Baseline (Slow Merge) vs. Enhanced (Zero-Merge)

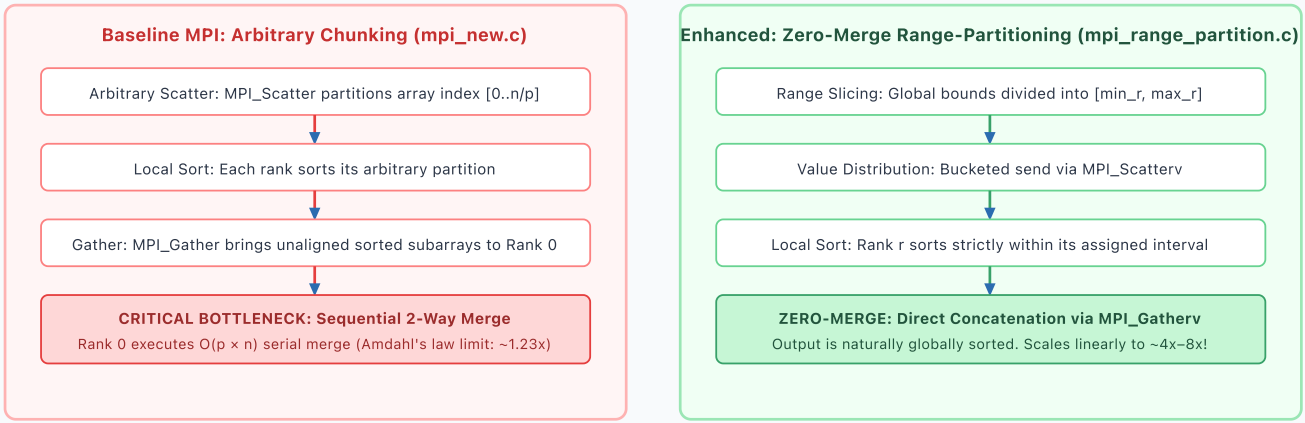


Figure 3: CUDA Scalable Many-Core GPU Architecture (cuda_scalable_pigeonhole.cu)



3. Summary of Additional Features & Improvements

- **Range-Partitioned Zero-Merge MPI:** Eliminates the serial 2-way merge bottleneck, allowing MPI to scale linearly with process count.
- **Multi-Block Scalable CUDA Engine:** Eliminates the 48 KB shared memory hardware barrier using global device atomics, supporting arbitrary dataset ranges (> 10,000,000).
- **Shared-Memory Multi-Core Parallelism (OpenMP):** Enables native multi-threaded benchmarking directly on multi-core MacBooks and Linux machines.
- **Dynamic Memory Safety:** Replaced unsafe stack-allocated VLAs with dynamic heap memory (`calloc`) to prevent segmentation faults.
- **Google Colab 1-Click Interactive Notebook:** Included `HPCS_Pigeonhole_Sort_Colab.ipynb` so users on MacBooks can run the entire suite (including NVIDIA CUDA GPUs) with zero local setup.